

Week 1 – Design Studio: Answers

Read this after you've done the studio. It is not the one right answer – it's what a strong step-1 answer contains, and where most first attempts go wrong. Note you will not need this level of detail for all future projects to save time.

Note — think about "the system should be accurate."

Accurate *for whom?* On *which* transactions? And *what happens* when it isn't?

If you can't answer those, the sentence isn't a requirement yet.

What to focus on

1. The users

"The bank" is not a user. People at the bank are, and they want different things from a flag:

WHO	WHAT THEY DO WITH A FLAG	WHAT THEY NEED FROM YOU
The cardholder	Gets declined, or gets a step-up text	Not to be embarrassed at dinner; a fast way to say "that was me"
The merchant	Loses the sale	A low false-decline rate on <i>their</i> traffic
The fraud analyst	Works a review queue	A queue they can actually clear; the trace, not just the verdict
The fraud ops manager	Owens the loss number and the analyst headcount	The trade-off made explicit
Compliance / disputes	Handles chargebacks, complaints, regulators	Auditability; a recourse path that exists

If your sketch named one of these, you're where most people start. A strong answer names most of them, because each one turns into a requirement.

2. The decision — "flag fraud" isn't one

The real decision is a **route** per transaction, and the routes are most of the design:

- **Approve** — the default.
- **Step-up** — challenge the cardholder (a text, an app prompt). Cheap, fast, and reversible.
- **Review** — hold it and send it to an analyst. Slow, expensive, bounded by headcount.
- **Decline** — refuse. Fast, and the most expensive route when it's wrong.

Notice what the model does in that list: nothing. The model produces a **score**. Code turns the score into a route, using thresholds that a person owns. *The model scores, the code routes*. That one sentence is your whole step 2 for this system, and it's an idea we'll name properly in Week 3 — the determinism boundary.

Here is a trap inside the trap: most sketches have exactly one action, "block." Step-up is the single biggest lever in real fraud systems, and almost nobody proposes it cold.

3. Should it be a model at all?

Partly no. A large share of fraud catch comes from **rules**: five transactions in a minute, a card in two countries within an hour, a known-bad merchant. Rules cost nothing to evaluate, nothing to explain, and the analysts already trust them. The model earns its place on what rules can't express — but it sits *behind* the rules, not instead of them. Asking this question is the difference between a good answer and a great one.

4. The three numbers

A design without these is a guess wearing a diagram. Pick numbers and commit; you can be wrong, but you can't be vague.

- **Scale**. A mid-size card portfolio might see 50 million transactions a day — about 600 a second on average, with peaks five to ten times that.
- **Latency**. The score is *in the authorization path*. The whole authorization has a budget of roughly a second and most of it is spent elsewhere. Your scoring call gets tens of milliseconds. That one number decides next week's architecture: a small model, features precomputed, anything slow running *after* the decision.
- **Cost**. Fractions of a cent per transaction, or the economics don't work at 50 million a day. The expensive path is analyst review — the review queue is a budget, expressed in people.

5. The rates

This is what step 1 actually produces. Tonight you could only approximate it; next week you get the exact form. Here is what it looks like:

- **Fraud caught** — some fraction of fraud *dollars*, and on which slices? Card-present versus card-not-present; new accounts versus old; high-value transactions. The aggregate number always hides the slice that's failing.
- **False declines** — the rate at which legitimate transactions are refused. The cardholder and the merchant care about this number more than any other, and the fraud team is the group least motivated to minimize it. So: *who owns it*?
- **Review volume** — flags per day have to fit the analysts. A model that flags 2% of 50 million transactions produces a million reviews a day. Nobody has that queue. The threshold is set by headcount, not by the ROC curve.
- **The remainder** — fraud that gets through becomes a chargeback, thirty to ninety days later. A legitimate transaction that gets blocked becomes a cardholder on the phone. *Both* paths need a design. Most sketches design neither.

6. The owner

who, in your company, decides the false-positive rate? In a real issuer it's usually the head of fraud operations, and it's a negotiated number — fraud loss against customer friction, a business decision made with technical information. If nobody owns it, then whoever set a threshold in a notebook at 2 a.m. owns it.

Two things strong answers mention

- **Labels arrive late, and biased.** Chargebacks take a month or three. And a transaction you *declined* never gets a label at all — you blocked it, so you never learn whether it was fraud. The system's own decisions shape tomorrow's training data. That's property two, and it comes back later weeks.
- **Fraudsters adapt.** The population you're serving is learning your system. Whatever works in March won't in June. That's property three and address later.

Check your sketch

Did it have — all the users, not one? Routes, not a single "block"? Step-up? Rules before the model? Numbers for scale, latency, and cost? At least one rate with a slice? A path for *both* kinds of mistake? A name next to the false-decline rate? A word about where labels come from?

If it had two of those, you're normal. It's saved now. We'll check this again at the end of the course.

Answer

This is what a strong sketch looks like when it is actually handed in.

Fraud flagging — step 1 sketch.

Users. Cardholder (wants: never declined at dinner; a 10-second way to say "that was me"). Merchant (wants: low false declines on their traffic; they eat the lost sale). Fraud analyst (wants: a queue they can clear by 5pm, and the *why* next to each case). Fraud ops manager (owns the loss number and the headcount). Disputes/compliance (wants: every decision traceable; a recourse path that exists on paper before a regulator asks).

The decision. Not "flag / don't flag." Per transaction, pick one of four routes: **approve** (default) · **step-up** (text or app prompt; cheap, reversible, gives us a label in seconds) · **review** (hold, send to analyst; slow, bounded by headcount) · **decline** (fast; the most expensive route when wrong). The model does not choose the route. The model emits a score; code maps score → route with thresholds a named person owns.

Three numbers.

- *Scale:* 50M transactions/day $\approx$ 600/s average, plan for 5,000/s peak (Black Friday, lunch hour).
- *Latency:* scoring is inside the authorization call. Auth budget $\sim$ 1s total, most spent elsewhere. **Score $\leq$ 30 ms p99.** Anything slower runs after the decision, not in it.
- *Cost:* compute $\leq$ \$0.0005/transaction ($\approx$ \$25k/day ceiling). The real budget is analysts: 40 analysts $\times$ 150 cases/day = **6,000 reviews/day**, which is 0.012% of traffic. That number sets the review threshold, not the ROC curve.

Rates (what "good" means, for whom).

RATE	TARGET	SLICE	WHEN WE MISS (REMAINDER)	OWNER
Fraud caught (by \$)	$\geq 70\%$	card-not-present; report separately for new accounts (< 30 days)	chargeback path, 30–90 days later; loss booked	Head of fraud ops
False declines	$\leq 0.1\%$ of legit txns	overall; $\leq 0.3\%$ on new accounts, tracked separately	cardholder confirms in-app in < 60s → auto-reversal; call center as backstop	Head of cardholder experience (not fraud — they aren't motivated to lower it)
Step-up rate	$\leq 2\%$ of txns, $\geq 90\%$ completed	—	abandoned step-up = declined; counts against false-decline number	Head of cardholder experience
Review volume	$\leq 6,000/\text{day}$	—	queue over capacity → raise review threshold, do not convert overflow to declines	Fraud ops manager
Approvals on the confirmed-stolen list	0 — invariant	—	not a rate; a bug. Handled by a rule, never the model	Fraud ops manager
Decisions without a stored trace	0 — invariant	—	compliance can't answer a dispute without one	Compliance

Trade-off, stated. Under pressure, fraud catch gives way, not customer friction: if the queue overflows we let more fraud through rather than decline more legit cardholders. Reg E / Reg Z dispute timelines and adverse-action explanations are requirements, not features — the trace and reason codes exist because of them.

What happens when it's wrong — both directions. Fraud approved → chargeback in 30–90 days → label arrives late, into training data. Legit declined → cardholder on the phone, *and no label ever* — we blocked it, so we never learn. Step-up is the fix for both: fast label, reversible mistake.

Step 2 (first half) — framing the AI task.

Should it be a model at all? Partly no. Rules first: hot list (stolen cards), velocity (> 5 txns/min), geo-impossible (two countries in an hour), known-bad merchants. Rules cost nothing, explain themselves, and analysts already trust them. The model scores what's left — the residual the rules can't express.

I/O contract. *In*: amount, merchant id + category code, country, channel (CP / CNP), card age, velocity counts at 1 min / 1 h / 24 h, device id, seconds since last transaction. *Freshness*: velocity $\leq$ 1 s stale; card and merchant profiles $\leq$ 24 h. *Out*: score $\in [0, 1]$, top-3 reason codes, model version. Schema, not free text.

Must not decide: the thresholds; anything on the hot list; whether a decline is final (the cardholder can always reverse via step-up); when to retrain.

Known unknowns. Labels are late and biased toward what we approved. Fraudsters adapt — March's model is not June's. Don't know yet what to do about either.